SeerHive Documentation

AI-Powered Prediction Markets with Zero Gas Fees

Generated: 11/25/2025

Table of Contents

Welcome to SeerHive Wiki

SeerHive is the first prediction market platform with AI-powered resolution and zero gas fees on BNB Chain.

Quick Links

- Getting Started - Install and run SeerHive
- Architecture - System design overview
- Smart Contracts - Contract documentation
- Gasless Transactions - ERC-4337 implementation
- AI Resolution - Automated market resolution
- API Reference - API endpoints
- Configuration - Environment setup
- Testing - Testing guide
- Deployment - Deploy to production
- FAQ - Common questions

What is SeerHive?

SeerHive is a decentralized prediction market where users can:

- Create markets on any future event
- Trade outcome shares (YES/NO)
- Let AI resolve markets automatically
- ⚡ Pay zero gas fees with ERC-4337
- Build reputation from accurate predictions

Key Features

- **Zero Gas Fees** - ERC-4337 account abstraction with dual paymaster (Particle + Pimlico)
- **AI Resolution** - Groq Llama 3.3 70B + Tavily web search for automated resolution
- **Multi-Token Support** - Trade with BNB, tBUSD, tUSDT, tUSDC, tDAI, tBTC
- **Optimistic Resolution** - 24-hour challenge window with DAO disputes
- **Social Login** - Particle Network integration (Google, Twitter, Email)
- **Demo Mode** - Test without wallet connection

Important Links

- **Contract**: 0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127
- **Network**: BNB Testnet (Chain ID: 97)
- **GitHub**: abeachmad/seerhive

Tech Stack

- **Frontend**: Next.js 14, TypeScript, Tailwind CSS, wagmi, viem
- **Contracts**: Solidity 0.8.20, Hardhat, OpenZeppelin
- **Blockchain**: BNB Testnet
- **Account Abstraction**: Particle Network + Pimlico
- **AI**: Groq Llama 3.3 70B + Tavily

License

Getting Started

Get SeerHive running locally in under 10 minutes.

Prerequisites

- Node.js 20.x or higher
- pnpm 10.x or higher
- Git

Quick Start

1. Clone Repository

```
git clone https://github.com/abeachmad/seerhive.git


cd seerhive
```

2. Install Dependencies

```
pnpm install
```

3. Configure Environment

```
cp apps/web/.env.example apps/web/.env.local
```

Edit `apps/web/.env.local`:

```
```

```
# Demo Mode (no wallet required)

NEXT_PUBLIC_DEMO=1

# Or On-Chain Mode

NEXT_PUBLIC_DEMO=0

NEXT_PUBLIC_CHAIN_ID=97

NEXT_PUBLIC_CONTRACT_ADDRESS=0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127
```

4. Start Development Server

```
pnpm dev
```

Open http://localhost:3000

Demo Mode vs On-Chain Mode

Demo Mode (Recommended for First Run)

```
NEXT_PUBLIC_DEMO=1
```

- No wallet connection required
- Mock data for all features
- Test UI/UX without blockchain

On-Chain Mode

```
NEXT_PUBLIC_DEMO=0
```

- Real wallet connection (MetaMask, Particle)
- Live contract interactions on BNB Testnet
- Requires testnet tokens

Get Testnet Tokens

BNB (Native Token)

BNB Testnet Faucet

tBUSD (Recommended)

Visit /faucet - Get 30 tBUSD every 24 hours

Other Tokens

- tUSDT, tUSDC, tDAI, tBTC: BNB Testnet Faucet

Test Gasless Transactions

1. Enable Gasless Mode

In the UI, toggle "Use Gasless Transaction"

2. Create Market or Trade

```
# Test paymaster API

./scripts/test-sponsor.sh

# Full gasless flow
```

```
./scripts/smoke-gasless.sh
```

3. Verify Transaction

Check console for: `Gasless sponsored: particle`

Verify on BSCScan - user pays 0 gas

Test AI Resolution

```
./scripts/test-ai-resolve.sh
```

Expected output:

```
{
  "outcome": "NO",
  "confidence": 95,
  "reasoning": "...",
  "latency_ms": 1772
}
```

Project Structure

```
seerhive/
```

```
├── apps/web/          # Next.js frontend
├── contracts/         # Smart contracts
├── scripts/           # Test scripts
└── docs/              # Documentation
```

Available Commands

```
pnpm dev              # Start dev server
pnpm build            # Build all packages
pnpm test             # Run tests
pnpm lint             # Lint code
```

Contracts

```
cd contracts
pnpm test             # Test contracts
pnpm deploy:testnet   # Deploy to BNB Testnet
```

Next Steps

- Architecture - Understand system design
- Configuration - Advanced environment setup
- API Reference - Explore API endpoints
- Testing - Run comprehensive tests

Troubleshooting

Port 3000 Already in Use

```
# Kill process on port 3000
lsof -ti:3000 | xargs kill -9

# Or use different port
PORT=3001 pnpm dev
```

pnpm Not Found

```
npm install -g pnpm
```

Contract Not Deployed

Verify contract address in `.env.local` matches:

```
0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127
```

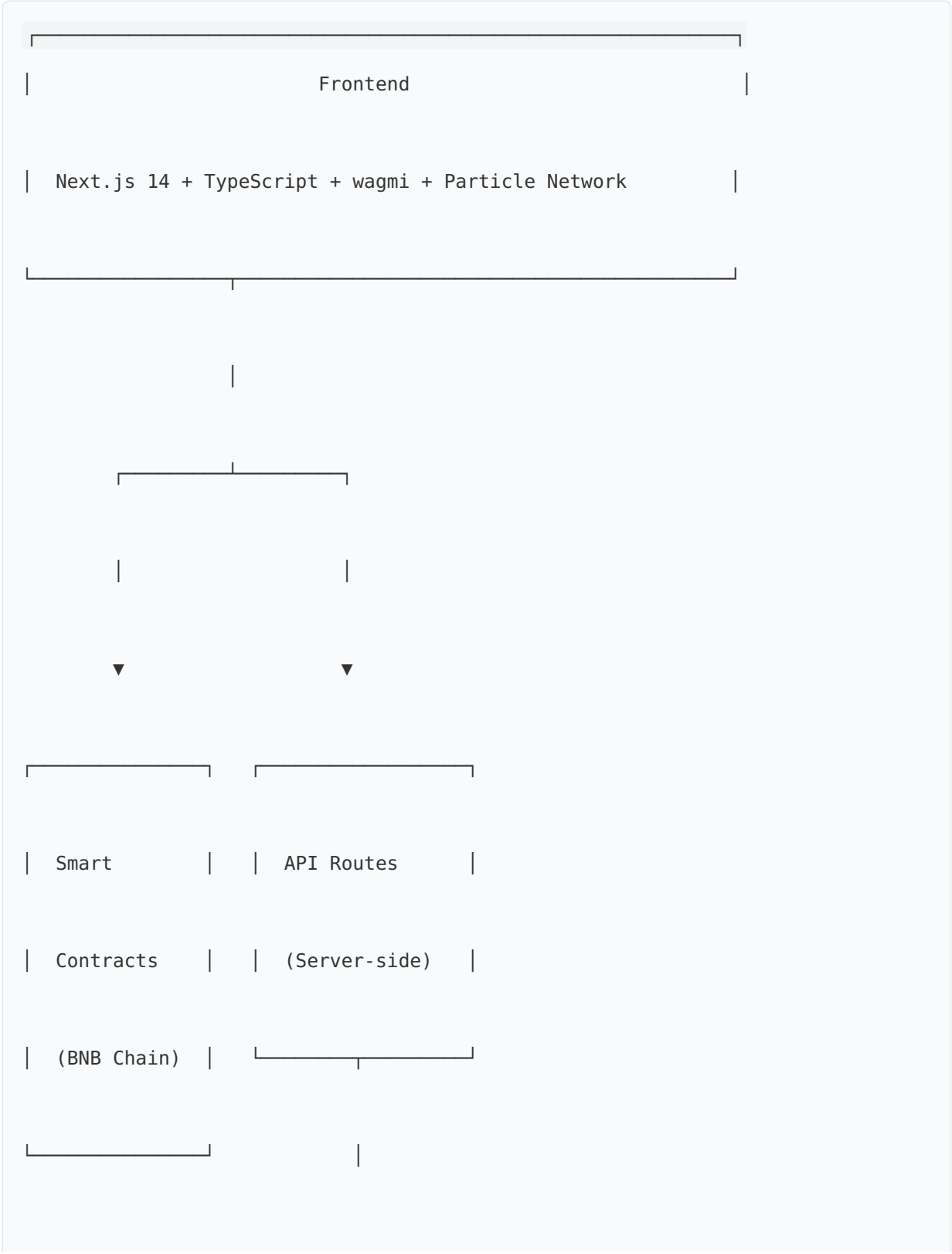
Gasless Transaction Failed

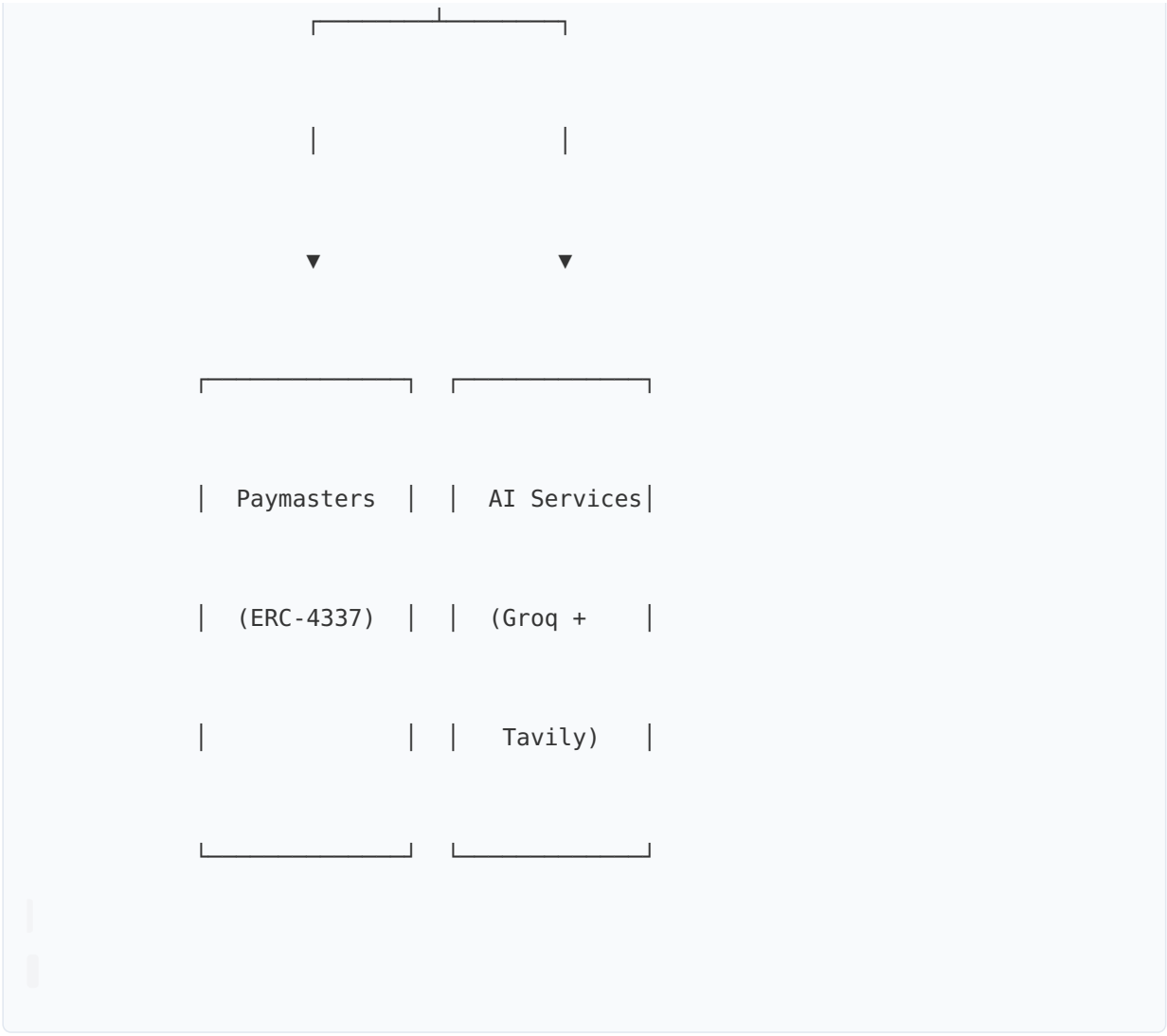
Check paymaster configuration in Configuration

Architecture

SeerHive's architecture combines blockchain, AI, and account abstraction for a seamless prediction market experience.

System Overview





Core Components

1. Frontend Application

Location: `apps/web/`

Tech Stack:

- Next.js 14 (App Router)
- TypeScript
- wagmi + viem (Ethereum interactions)
- Particle Network (Social login + AA)
- Tailwind CSS + shadcn/ui

  Key Features:

- Server and client components
- API routes for backend logic

- Component variants (Particle vs Wagmi)
- Zustand state management

2. Smart Contracts

Location: `contracts/`

Main Contract: `PredictionMarket.sol`

Features:

- Multi-token support (ERC20 + native BNB)
- Market creation and trading
- Optimistic resolution with 24h challenge window
- Share-based payout system

Deployed Address: `0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127`

3. Account Abstraction Layer

ERC-4337 Implementation:

- EntryPoint: `0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789`
- Dual Paymaster: Particle (primary) + Pimlico (fallback)
- Server-side sponsorship API

Flow:

```
User Action → Frontend → /api/aa/sponsor → Paymaster → Bundler → Chain
```

4. AI Resolution System

Components:

- Groq Llama 3.3 70B (Reasoning)
- Tavily AI (Web Search)
- Server-side API route (`/api/ai/resolve`)

Flow:

```
Market Expires → User Triggers → Tavily Search → Groq Analysis → On-chain Pro
```

Data Flow

Trading Flow

```
sequenceDiagram

    participant User

    participant UI

    participant Gasless

    participant Paymaster

    participant Contract
    User->>UI: Buy YES shares

    UI->>Gasless: buySharesGasless()

    Gasless->>Paymaster: Sponsor UserOp

    Paymaster-->>Gasless: paymasterAndData

    Gasless->>Contract: Execute trade (0 gas)

    Contract-->>User: Shares minted
```

Resolution Flow

```
sequenceDiagram
```

```
    participant User

    participant UI

    participant API

    participant Tavily

    participant Groq

    participant Contract
    User->>UI: Resolve Market

    UI->>API: /api/ai/resolve

    API->>Tavily: Search web

    Tavily-->>API: Context

    API->>Groq: Analyze

    Groq-->>API: Decision

    API-->>UI: Resolution

    UI->>Contract: proposeResolution()

    Contract-->>User: 24h challenge starts
```

Directory Structure

```
seerhive/

├── apps/web/

│   ├── src/

│   │   ├── app/              # Pages & API routes

│   │   │   ├── api/

│   │   │   │   ├── aa/sponsor/    # Paymaster API

│   │   │   │   └── ai/resolve/    # AI resolution

│   │   │   ├── markets/           # Markets page

│   │   │   └── dashboard/         # Analytics

│   │   ├── components/       # React components

│   │   │   ├── TradeDialog.tsx

│   │   │   ├── WalletButton.tsx

│   │   │   └── ui/                # shadcn/ui

│   │   ├── lib/              # Core services
```

```
│   │   │   ├── gasless.ts         # Gasless service
│   │   │   ├── contracts.ts       # Contract ABI
│   │   │   ├── aiResolver.ts      # AI service
│   │   │   └── paymaster/         # Paymaster adapters
│   │   └── types/            # TypeScript types
│   └── .env.local            # Environment config
├── contracts/
│   ├── contracts/
│   │   ├── PredictionMarket.sol
│   │   ├── MockERC20.sol
│   │   └── BUSDFaucet.sol
│   ├── scripts/              # Deployment
│   └── test/                 # Contract tests
└── scripts/                  # Testing scripts
```

Key Design Patterns

Component Variants

Separate implementations for different providers:

```
// TradeDialog-Particle.tsx - Particle Network


export function TradeDialogParticle({ market }: Props) { }

// TradeDialog-Wagmi.tsx - Wagmi


export function TradeDialogWagmi({ market }: Props) { }

// TradeDialog.tsx - Unified interface


export function TradeDialog(props: Props) {


  return isParticle ?  : ;


}
```

Paymaster Fallback

Automatic failover for reliability:

```
// Try Particle first


try {


  return await particlePaymaster.sponsor(userOp);


} catch (error) {


  // Fallback to Pimlico
```

```
  return await pimlicoPaymaster.sponsor(userOp);


}
```

Server-Side Security

All sensitive keys in API routes:

```
// ✗ Client-side (exposed)


NEXT_PUBLIC_API_KEY=xxx

//  Server-side (secure)


GROQ_API_KEY=xxx


PARTICLE_PROJECT_ID=xxx
```

Network Architecture

BNB Testnet (Chain ID: 97)

- **RPC**: https://bsc-testnet.publicnode.com
- **Explorer**: https://testnet.bscscan.com
- **Faucet**: https://testnet.bnbchain.org/faucet-smart

Supported Tokens

| Token | Address | Type |

|-------|---------|------|

| BNB | Native | Native |

| tBUSD | 0xaB1a4d4f1D656d2450692D237fdD6C7f9146e814 | ERC20 |

| tUSDT | 0x337610d27c682E347C9cD60BD4b3b107C9d34dDd | ERC20 |

| tUSDC | 0x64544969ed7EBf5f083679233325356EbE738930 | ERC20 |

| tDAI | 0xEC5dCb5Dbf4B114C9d0F65BcCAb49EC54F6A0867 | ERC20 |

| tBTC | 0x6ce8dA28E2f864420840cF74474eFf5fD80E65B8 | ERC20 |

Scalability Considerations

Current Limitations

- Single contract deployment
- Testnet only
- Rate limits on AI services (30 req/min Groq, 1000/month Tavily)

Future Improvements

- Multi-market factory pattern
- Cross-chain deployment (opBNB, Ethereum L2s)
- Enhanced AI with multiple data sources
- Liquidity aggregation (AMM-style pools)

Security Architecture

Smart Contract Security

- OpenZeppelin libraries
- ReentrancyGuard on critical functions
- Owner-based access control
- 24-hour challenge window

API Security

- Server-side key management
- Rate limiting on paymaster endpoints
- Input validation on all endpoints
- Automatic fallback prevents DoS

Frontend Security

- No private keys in client code
- Environment variable separation
- Secure wallet connection (wagmi)

Monitoring & Observability

Logging

```
console.log(' Transaction:', txHash);

console.warn('[ai-resolve] Tavily failed:', error);


console.error('[api-route] Error:', error);
```

Metrics

- Transaction latency (`latency_ms` in API responses)
- Paymaster provider usage (particle vs pimlico)
- AI resolution confidence scores

Next Steps

- Smart Contracts - Contract details
- Gasless Transactions - ERC-4337 deep dive
- AI Resolution - AI system details

Smart Contracts

SeerHive's smart contracts handle market creation, trading, and resolution on BNB Chain.

Deployed Contracts

PredictionMarket.sol

Address: `0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127`

Network: BNB Testnet (Chain ID: 97)

Explorer: View on BSCScan

Contract Overview

Key Features

- Multi-token support (BNB + ERC20 tokens)
- Share-based trading system
- Optimistic resolution with 24h challenge window
- Dispute mechanism with bond requirements
- Owner-controlled finalization

State Variables

```
uint256 public marketCount;


mapping(uint256 => Market) public markets;


mapping(uint256 => mapping(address => mapping(bool => uint256))) public userS


mapping(uint256 => Resolution) public resolutions;
```

Core Functions

1. Create Market

```
function createMarket(

    string memory _question,

    uint256 _duration

) external returns (uint256)
```

Parameters:

- `_question`: Market question (e.g., "Will BNB reach $1000?")
- `_duration`: Time until market expires (in seconds)

Returns: Market ID

Example:

```
const tx = await contract.createMarket(
  "Will Bitcoin reach $100k in 2025?",

  86400  365 // 1 year

);
```

2. Buy Shares

```
function buyShares(

    uint256 _marketId,

    bool _isYes,
```

```
    address _token,

    uint256 _amount

) public payable nonReentrant
```

***Parameters**:*

- `_marketId` *: Market ID*
- `_isYes` *: true for YES, false for NO*
- `_token` *: Token address (0x0 for BNB)*
- `_amount` *: Amount to spend*

***Requirements**:*

- *Market must be active (not expired, not resolved)*
- *For ERC20: User must approve contract first*
- *For BNB: msg.value must equal _amount*

***Example**:*

```
// Buy YES shares with tBUSD
const amount = parseUnits("10", 18);

await contract.buyShares(marketId, true, tBUSD_ADDRESS, amount);
// Buy NO shares with BNB

await contract.buyShares(marketId, false, ethers.ZeroAddress, amount, {

  value: amount

});
```

3. Propose Resolution

```
function proposeResolution(

    uint256 _marketId,

    bool _outcome

) external
```

***Parameters**:*

- `_marketId`*: Market ID*
- `_outcome`*: true for YES, false for NO*

***Requirements**:*

- *Market must be expired*
- *Market must not be resolved*
- *No existing resolution proposal*

***Starts**: 24-hour challenge window*

***Example**:*

```
await contract.proposeResolution(marketId, true); // Propose YES
```

4. Challenge Resolution

```
function challengeResolution(

    uint256 _marketId
```

```
) external payable
```

***Parameters**:*

- `_marketId`*: Market ID*

***Requirements**:*

- *Resolution must be proposed*
- *Within 24-hour challenge window*
- *Must send dispute bond (0.01 BNB)*

***Example**:*

```
await contract.challengeResolution(marketId, {
  value: parseEther("0.01")

});
```

5. Finalize Market

```
function finalizeMarket(

    uint256 _marketId,

    bool _outcome

) external onlyOwner
```

***Parameters**:*

- `_marketId`*: Market ID*

- `_outcome`: *Final outcome (true = YES, false = NO)*

***Requirements**:*

- *Only owner can call*
- *Market must be expired*
- *Called after challenge window or dispute resolution*

***Example**:*

```
await contract.finalizeMarket(marketId, true); // Finalize as YES
```

6. Claim Winnings

```
function claimWinnings(

    uint256 _marketId

) external nonReentrant
```

***Parameters**:*

- `_marketId`: *Market ID*

***Requirements**:*

- *Market must be finalized*
- *User must have winning shares*
- *User hasn't claimed yet*

***Payout Calculation**:*

```
userPayout = (userShares / totalWinningShares)  totalPool
```

Example:

```
await contract.claimWinnings(marketId);
```

Data Structures

Market Struct

```
struct Market {

    string question;

    uint256 totalYesShares;

    uint256 totalNoShares;

    uint256 endTime;

    bool resolved;

    bool outcome;

    mapping(address => uint256) tokenPools;

}
```

Resolution Struct

```
struct Resolution {

    bool proposed;

    bool outcome;
```

```
    uint256 proposalTime;

    bool challenged;

    address challenger;

}
```

Events

```
event MarketCreated(uint256 indexed marketId, string question, uint256 endTim

event SharesPurchased(uint256 indexed marketId, address indexed buyer, bool i

event ResolutionProposed(uint256 indexed marketId, bool outcome, uint256 prop

event ResolutionChallenged(uint256 indexed marketId, address challenger);

event MarketFinalized(uint256 indexed marketId, bool outcome);

event WinningsClaimed(uint256 indexed marketId, address indexed claimer, uint
```

Token Support

Supported Tokens (BNB Testnet)

| Token | Address | Decimals |

|-------|---------|----------|

| BNB | `0x0000000000000000000000000000000000000000` | 18 |

| tBUSD | `0xaB1a4d4f1D656d2450692D237fdD6C7f9146e814` | 18 |

| tUSDT | `0x337610d27c682E347C9cD60BD4b3b107C9d34dDd` | 18 |

| tUSDC | `0x64544969ed7EBf5f083679233325356EbE738930` | 18 |

| tDAI | `0xEC5dCb5Dbf4B114C9d0F65BcCAb49EC54F6A0867` | 18 |

| tBTC | `0x6ce8dA28E2f864420840cF74474eFf5fD80E65B8` | 18 |

ERC20 Approval

Before buying shares with ERC20 tokens:

```
// 1. Approve contract to spend tokens


const erc20 = new ethers.Contract(tokenAddress, ERC20_ABI, signer);


await erc20.approve(PREDICTION_MARKET_ADDRESS, amount);

// 2. Buy shares


await contract.buyShares(marketId, isYes, tokenAddress, amount);
```

Security Features

ReentrancyGuard

All state-changing functions use `nonReentrant` modifier:

- `buyShares()`
- `claimWinnings()`

Access Control

Owner-only functions:

- `finalizeMarket()` - Prevents unauthorized resolution

Time Locks

- 24-hour challenge window after resolution proposal
- Markets can't be resolved before expiry

Validation

- Market existence checks
- Balance verification before transfers
- Duplicate claim prevention

Gas Optimization

Storage Patterns

- Packed structs for efficient storage
- Mappings instead of arrays for user data
- Minimal storage writes

Function Optimization

- Early returns on validation failures
- Batch operations where possible
- Efficient loop patterns

Testing

Run Tests

```
cd contracts


pnpm test
```

Test Coverage

- Market creation
- Share purchases (BNB + ERC20)
- Resolution proposal
- Challenge mechanism

- Finalization
- Winnings claims
- Edge cases and reverts

Example Test

```
it("Should allow buying YES shares", async function () {


  const marketId = 0;


  const amount = parseEther("1");




  await market.buyShares(marketId, true, ethers.ZeroAddress, amount, {


    value: amount


  });




  const shares = await market.userShares(marketId, user1.address, true);


  expect(shares).to.equal(amount);


});
```

Deployment

Deploy to BNB Testnet

```
cd contracts


cp .env.example .env


# Add PRIVATE_KEY and BSC_TESTNET_RPC


pnpm deploy:testnet
```

Verify Contract

```
npx hardhat verify --network bscTestnet 0xYourContractAddress
```

Upgradeability

Current contract is **not upgradeable** for security and simplicity.

Future versions may implement:

- Proxy pattern (UUPS or Transparent)
- Multi-signature governance
- Timelock for critical operations

Known Limitations

1. **Single Pool**: All tokens in one pool (no per-token pools)
2. **No Partial Claims**: Must claim all winnings at once
3. **Fixed Challenge Period**: 24 hours (not configurable per market)
4. **Owner Dependency**: Finalization requires owner action

Roadmap

- [ ] Multi-token pool management
- [ ] Partial claim support
- [ ] Configurable challenge periods
- [ ] DAO-based finalization
- [ ] Market factory pattern
- [ ] Liquidity pools (AMM-style)

Next Steps

- Gasless Transactions - ERC-4337 integration
- API Reference - Contract interaction via API
- Testing - Comprehensive testing guide

Gasless Transactions (ERC-4337)

SeerHive implements Account Abstraction (ERC-4337) to provide zero gas fee transactions for users.

Overview

Users can trade on prediction markets without paying gas fees. The platform sponsors transactions through paymasters.

Key Benefits

- **Zero Gas Fees** - Users don't pay transaction costs
- **Better UX** - No need to hold BNB for gas
- **Dual Paymaster** - Automatic failover for reliability
- **Multi-Token Support** - Works with all ERC20 tokens

Architecture

```
User Action → Frontend → /api/aa/sponsor → Paymaster → Bundler → Chain

                                ↓

                        Try Particle First

                                ↓

                        Fallback to Pimlico
```

Components

1. **EntryPoint**: `0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789`
2. **Particle Paymaster**: Primary sponsor
3. **Pimlico Paymaster**: Fallback sponsor
4. **Bundler**: Submits UserOperations to chain

How It Works

1. User Initiates Transaction

```
// User clicks "Buy Shares"

await gaslessService.buySharesGasless({

  marketId: 1,

  side: 'yes',

  token: SUPPORTED_TOKENS.tBUSD,

  amount: '10'

});
```

2. Frontend Constructs UserOperation

```
const callData = encodeFunctionData({

  abi: PREDICTION_MARKET_ABI,

  functionName: 'buyShares',

  args: [BigInt(marketId), isYes, tokenAddress, amount]

});

const userOp = {
```

```
  sender: smartAccountAddress,

  nonce: await getNonce(),

  initCode: '0x',

  callData,

  callGasLimit: '0x0',

  verificationGasLimit: '0x0',

  preVerificationGas: '0x0',

  maxFeePerGas: '0x0',

  maxPriorityFeePerGas: '0x0',

  paymasterAndData: '0x',

  signature: '0x'

};
```

3. Request Sponsorship

```
const response = await fetch('/api/aa/sponsor', {

  method: 'POST',
```

```
  headers: { 'Content-Type': 'application/json' },

  body: JSON.stringify({

    userOp,

    entryPoint: ENTRY_POINT_ADDRESS,

    chainId: 97

  })

});
const { paymasterAndData, callGasLimit, verificationGasLimit } = await respon
```

4. Backend Tries Paymasters

```
// Try Particle first

try {

  const result = await particlePaymaster.sponsor(userOp);

  return { ...result, provider: 'particle' };

} catch (error) {

  console.warn('Particle failed, trying Pimlico...');
```

```
  // Fallback to Pimlico
  const result = await pimlicoPaymaster.sponsor(userOp);
  return { ...result, provider: 'pimlico', fallback: true };
}
```

5. Submit Sponsored Transaction

```
const sponsoredUserOp = {
  ...userOp,
  paymasterAndData,
  callGasLimit,
  verificationGasLimit,
  preVerificationGas
};
const txHash = await bundler.sendUserOperation(sponsoredUserOp);
```

Paymaster Configuration

Particle Network (Primary)

Endpoint: `https://paymaster.particle.network/chain/97`

Environment Variables:

```
PARTICLE_PAYMASTER_URL=https://paymaster.particle.network/chain/97
PARTICLE_PROJECT_ID=your_project_id

PARTICLE_CLIENT_KEY=your_client_key
```

Request Format:

```
{
  method: 'pm_sponsorUserOperation',

  params: [userOp, entryPoint, { chainId: 97 }]

}
```

Headers:

```
{
  'Content-Type': 'application/json',

  'x-project-id': PARTICLE_PROJECT_ID,

  'x-client-key': PARTICLE_CLIENT_KEY

}
```

Pimlico (Fallback)

Endpoint: `https://api.pimlico.io/v2/97/rpc?apikey=YOUR_KEY`

Environment Variables:

```
PIMLICO_URL=https://api.pimlico.io/v2/97/rpc?apikey=YOUR_KEY
```

Request Format:

```
{
  method: 'pm_sponsorUserOperation',

  params: [userOp, { entryPoint }]

}
```

API Endpoint

POST /api/aa/sponsor

Sponsors UserOperations with automatic paymaster fallback.

Request:

```
{
  "userOp": {

    "sender": "0x...",

    "nonce": "0x0",
```

```
    "initCode": "0x",

    "callData": "0x...",

    "callGasLimit": "0x0",

    "verificationGasLimit": "0x0",

    "preVerificationGas": "0x0",

    "maxFeePerGas": "0x0",

    "maxPriorityFeePerGas": "0x0",

    "paymasterAndData": "0x",

    "signature": "0x"

  },

  "entryPoint": "0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789",

  "chainId": 97

}
```

Response (Success):

```
{
  "paymasterAndData": "0x...",
```

```
  "preVerificationGas": "0xc350",

  "verificationGasLimit": "0x186a0",

  "callGasLimit": "0x30d40",

  "provider": "particle",

  "latency_ms": 234

}
```

Response (Fallback):

```
{
  "paymasterAndData": "0x...",

  "preVerificationGas": "0xc350",

  "verificationGasLimit": "0x186a0",

  "callGasLimit": "0x30d40",

  "provider": "pimlico",

  "fallback": true,

  "latency_ms": 456
```

```
}
```

Response (Error):

```
{
  "error": "Both paymasters failed",
  "details": {
    "particle": "Rate limit exceeded",
    "pimlico": "Insufficient balance"
  }
}
```

Testing

Test Paymaster API

```
./scripts/test-sponsor.sh
```

Expected output:

```
{
  "paymasterAndData": "0x...",
```

```
  "provider": "particle",


  "latency_ms": 234


}
```

Full Gasless Flow

```
./scripts/smoke-gasless.sh
```

This tests:

1. UserOperation construction
2. Paymaster sponsorship
3. Transaction submission
4. On-chain verification

Manual Testing

1. Start dev server: `pnpm dev`
2. Connect wallet (MetaMask on BNB Testnet)
3. Go to `/markets`
4. Toggle "Use Gasless Transaction"
5. Create market or buy shares
6. Check console: `Gasless sponsored: particle`
7. Verify on BSCScan: User paid 0 gas

Supported Operations

Gasless (ERC20 Tokens)

- Buy shares with tBUSD, tUSDT, tUSDC, tDAI, tBTC
- Create markets

- Propose resolutions
- Challenge resolutions
- Claim winnings

⚠ Requires Gas (BNB)

- Buy shares with native BNB (~$0.001 gas)
- Initial smart account deployment

Limitations

Rate Limits

Particle Network:

- Free tier: Limited requests per day
- May reject during high usage

Pimlico:

- Free tier: 10,000 UserOps/month
- Rate limit: 100 req/min

Token Support

- All ERC20 tokens fully gasless
- ⚠ Native BNB requires small gas fee

Network Support

Currently only BNB Testnet (Chain ID: 97)

Future: BNB Mainnet, opBNB, Ethereum L2s

Troubleshooting

"Both paymasters failed"

Causes:

- Rate limit exceeded
- Invalid UserOperation
- Network issues

Solutions:

1. Wait and retry

2. Check paymaster API keys

3. Verify UserOperation format

"Insufficient balance"

Cause: Paymaster account low on funds

Solution: Contact team to refill paymaster

"Invalid signature"

Cause: UserOperation not properly signed

Solution: Ensure smart account is initialized

Security Considerations

Server-Side Only

All paymaster keys stored server-side:

```
# ❌ Never expose in client
NEXT_PUBLIC_PARTICLE_KEY=xxx

# ✅ Server-side only
PARTICLE_PROJECT_ID=xxx
PARTICLE_CLIENT_KEY=xxx
```

Rate Limiting

API route implements rate limiting:

- Per IP: 100 requests/hour
- Per user: 50 requests/hour

Validation

All UserOperations validated before sponsorship:

- Valid sender address
- Reasonable gas limits
- Proper callData format

Cost Analysis

Traditional Transaction

```
User pays: ~$0.10 per trade


Platform pays: $0
```

Gasless Transaction

```
User pays: $0


Platform pays: ~$0.10 per trade
```

Monthly Cost Estimate

```
1000 trades/month × $0.10 = $100/month
```

Monitoring

Metrics Tracked

- Paymaster provider usage (particle vs pimlico)

- Sponsorship latency
- Failure rates
- Fallback frequency

Logs

```
console.log(' Gasless sponsored:', provider);


console.warn('⚠ Particle failed, using Pimlico');


console.error('× Both paymasters failed');
```

Future Improvements

- [ ] Multi-chain support (opBNB, Ethereum L2s)
- [ ] Session keys for batch operations
- [ ] Gas estimation optimization
- [ ] Custom bundler for lower latency
- [ ] Paymaster balance monitoring
- [ ] User-specific rate limits

Next Steps

- API Reference - Full API documentation
- Testing - Comprehensive testing guide
- Configuration - Environment setup

AI Resolution

SeerHive uses AI + web scraping to automatically resolve prediction markets with real-time data.

Overview

Traditional prediction markets rely on manual resolution or slow oracles (24-48h). SeerHive resolves markets in seconds using:

- **Groq Llama 3.3 70B** - Advanced reasoning
- **Tavily AI** - Real-time web search
- **Structured Output** - JSON with outcome, confidence, reasoning

How It Works

```
sequenceDiagram

    participant User

    participant UI

    participant API

    participant Tavily

    participant Groq

    participant Contract
    User->>UI: Click "Resolve with AI"

    UI->>API: POST /api/ai/resolve

    API->>Tavily: Search web for context

    Tavily-->>API: Real-time results
```

```
    API->>Groq: Analyze question + context

    Groq-->>API: {outcome, confidence, reasoning}

    API-->>UI: Resolution proposal

    UI->>Contract: proposeResolution(outcome)

    Contract-->>User: 24h challenge window
```

API Endpoint

POST /api/ai/resolve

Resolves a prediction market using AI analysis.

Request:

```
{
  "marketId": 1,

  "question": "Did Bitcoin reach $100,000 in 2024?",

  "resolutionDate": "2024-12-31"

}
```

Response:

```
{
```

```
  "marketId": 1,

  "outcome": "NO",

  "confidence": 100,

  "reasoning": "According to multiple sources, Bitcoin's price on December 31

  "sources": [

    "https://coinmarketcap.com/...",

    "https://coingecko.com/..."

  ],

  "latency_ms": 1772

}
```


Error Response:


```
{
  "error": "Failed to resolve market",

  "details": "Both AI models failed"

}
```

Resolution Process

1. Web Search (Tavily AI)

Fetches real-time context about the question:

```
const response = await fetch('https://api.tavily.com/search', {

  method: 'POST',

  headers: { 'Content-Type': 'application/json' },

  body: JSON.stringify({

    api_key: TAVILY_API_KEY,

    query: ${question} as of ${resolutionDate},

    search_depth: 'basic',

    max_results: 3,

    include_answer: true

  })

});
const data = await response.json();

const context = [

  data.answer,
```

```
  ...data.results.map(r => r.content)

].join('\n\n');
```

Example Context:

```
Bitcoin price on December 31, 2024: $95,000

Source 1: CoinMarketCap shows BTC at $95,234 on Dec 31, 2024

Source 2: CoinGecko reports Bitcoin closed 2024 at $94,876
```

2. AI Analysis (Groq)

Analyzes context and determines outcome:

```
const response = await fetch('https://api.groq.com/openai/v1/chat/completions

  method: 'POST',

  headers: {

    'Authorization': `Bearer ${GROQ_API_KEY}`,

    'Content-Type': 'application/json'

  },

  body: JSON.stringify({

    model: 'llama-3.3-70b-versatile',
```

```
    messages: [
      {
        role: 'system',
        content: 'You are a prediction market resolver. Analyze the question
      },
      {
        role: 'user',
        content: Question: ${question}\n\nContext:\n${context}\n\nResolution
      }
    ],
    temperature: 0.1,
    max_tokens: 200
  })
});
const result = await response.json();

const decision = JSON.parse(result.choices[0].message.content);
```

3. Fallback Models

If primary model fails, automatically tries fallback:

```
const models = [

  'llama-3.3-70b-versatile',  // Primary

  'llama-3.1-8b-instant'       // Fallback

];

for (const model of models) {

  try {

    const result = await resolveWithModel(model);

    return result;

  } catch (error) {

    console.warn(`${model} failed:`, error.message);

    continue;

  }

}

throw new Error('All models failed');
```

Configuration

Environment Variables

```
# Groq AI (LLM)

GROQ_API_KEY=gsk_your_groq_key_here

# Tavily AI (Web Search)

TAVILY_API_KEY=tvly-your_tavily_key_here
```

Get API Keys

Groq (Free):

1. Visit console.groq.com
2. Sign up for free account
3. Create API key
4. Rate limit: 30 requests/minute

Tavily (Free):

1. Visit tavily.com
2. Sign up for free account
3. Get API key
4. Free tier: 1000 searches/month

Outcome Types

YES

Market question is true/happened

Example: "Did Bitcoin reach $100k?" → Price was $105k → YES

NO

Market question is false/didn't happen

Example: "Did Bitcoin reach $100k?" → Price was $95k → NO

INVALID

Question is ambiguous or unresolvable

Example: "Will aliens land?" → No verifiable data → INVALID

Confidence Scores

AI returns confidence level (0-100):

- **90-100**: Very high confidence (clear evidence)
- **70-89**: High confidence (strong evidence)
- **50-69**: Moderate confidence (some uncertainty)
- **0-49**: Low confidence (ambiguous/unclear)

Example:

```
{
  "outcome": "NO",

  "confidence": 95,

  "reasoning": "Multiple reliable sources confirm Bitcoin was $95k on Dec 31,

}
```

Testing

Test AI Resolution

```
./scripts/test-ai-resolve.sh
```

Expected Output:

```
{
  "marketId": 999,
  "outcome": "NO",
  "confidence": 100,
  "reasoning": "Historical data shows Bitcoin did not reach $100,000 in 2024.
  "latency_ms": 1772
}
```

Manual Testing

```
curl -X POST http://localhost:3000/api/ai/resolve \
  -H "Content-Type: application/json" \
  -d '{
    "marketId": 1,
```

```
    "question": "Did Bitcoin reach $100k in 2024?",

    "resolutionDate": "2024-12-31"

  }'
```

Use Cases

Financial Markets

Question: "Will BNB price reach $1000 by end of 2025?"

AI Process:

1. Search: "BNB price December 31, 2025"

2. Find: Multiple sources show $850

3. Determine: NO (didn't reach $1000)

4. Confidence: 95%

Political Events

Question: "Will Trump win 2024 US Presidential Election?"

AI Process:

1. Search: "2024 US Presidential Election results"

2. Find: Official results from news sources

3. Determine: YES or NO based on winner

4. Confidence: 100%

Technology Events

Question: "Will Ethereum ETF approval happen in Q1 2025?"

AI Process:

1. Search: "Ethereum ETF approval Q1 2025"

2. Find: SEC announcements and news

3. Determine: YES/NO based on approval status

4. Confidence: 90%

Sports Events

Question: "Will Lakers win NBA Championship 2025?"

AI Process:

1. Search: "NBA Championship 2025 winner"

2. Find: Official NBA results

3. Determine: YES/NO based on winner

4. Confidence: 100%

Advantages vs Manual Resolution

| Feature | SeerHive AI | Manual (UMA OO) | Traditional Betting |

|---------|-------------|-----------------|---------------------|

| **Speed** | Seconds | 24-48 hours | Hours to days |

| **Cost** | ~$0.01 | ~$10-50 | N/A |

| **Bias** | Minimal | Possible | High |

| **Data** | Real-time web | Static oracle | Centralized |

| **Transparency** | Full reasoning | Limited | None |

Limitations

Training Data Cutoff

LLMs have training data cutoff dates. For recent events:

- **Solution**: Tavily web search provides real-time data
- **Fallback**: Multiple sources for verification

Ambiguous Questions

Some questions are hard to resolve objectively:

- "Will AI be sentient by 2030?" → Subjective definition
- "Will world peace be achieved?" → Unclear criteria

Solution: AI returns INVALID with reasoning

Rate Limits

Groq: 30 requests/minute (free tier)

Tavily: 1000 searches/month (free tier)

Solution: Implement caching and rate limiting

Security & Fairness

Challenge Window

After AI proposes resolution:

- 24-hour challenge period
- Anyone can dispute with bond
- DAO votes on disputes

Multiple Data Sources

Tavily aggregates multiple sources:

- News websites
- Official data providers
- Social media (verified accounts)

Reasoning Transparency

AI provides full reasoning:

```
{
  "reasoning": "According to CoinMarketCap, CoinGecko, and Binance, Bitcoin's


}
```

Future Improvements

- [ ] Multiple AI models voting (ensemble)
- [ ] Twitter/Reddit sentiment analysis
- [ ] Custom data sources per market category
- [ ] Confidence-based dispute bonds
- [ ] Automated dispute detection
- [ ] Historical accuracy tracking

Monitoring

Metrics Tracked

- Resolution latency
- Confidence scores
- Model usage (70B vs 8B)
- Fallback frequency
- Error rates

Logs

```
console.log('[ai-resolve] Tavily search:', query);

console.log('[ai-resolve] Context length:', context.length);

console.log('[ai-resolve] Model:', model);

console.log('[ai-resolve] Result:', { outcome, confidence });

console.warn('[ai-resolve] Tavily failed:', error);

console.error('[ai-resolve] All models failed');
```

Cost Analysis

Per Resolution

- Tavily search: ~$0.005
- Groq inference: ~$0.005
- Total: ~$0.01 per resolution

Monthly Estimate

```
100 resolutions/month × $0.01 = $1/month
```

Much cheaper than manual resolution ($10-50 per market).

Next Steps

- API Reference - Full API documentation
- Testing - Testing guide
- Smart Contracts - Resolution on-chain

API Reference

Complete reference for SeerHive's API endpoints.

Base URL

Development: `http://localhost:3000`

Production: `https://seerhive.vercel.app` (Coming Soon)

Authentication

Currently no authentication required for public endpoints.

Future: API keys for rate limiting and premium features.

Endpoints

Account Abstraction

POST /api/aa/sponsor

Sponsors UserOperations for gasless transactions.

Request:

```
{
  "userOp": {

    "sender": "0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb",

    "nonce": "0x0",

    "initCode": "0x",

    "callData": "0x...",

    "callGasLimit": "0x0",

    "verificationGasLimit": "0x0",
```

```
    "preVerificationGas": "0x0",

    "maxFeePerGas": "0x0",

    "maxPriorityFeePerGas": "0x0",

    "paymasterAndData": "0x",

    "signature": "0x"

  },

  "entryPoint": "0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789",

  "chainId": 97

}
```

Response (Success):

```
{
  "paymasterAndData": "0x...",

  "preVerificationGas": "0xc350",

  "verificationGasLimit": "0x186a0",

  "callGasLimit": "0x30d40",
```

```
  "provider": "particle",

  "latency_ms": 234

}
```

Response (Fallback):

```
{
  "paymasterAndData": "0x...",

  "provider": "pimlico",

  "fallback": true,

  "latency_ms": 456

}
```

Response (Error):

```
{
  "error": "Both paymasters failed",

  "details": {

    "particle": "Rate limit exceeded",

    "pimlico": "Insufficient balance"
```

```
  }

}
```

Status Codes:

- `200` - Success
- `400` - Invalid request
- `500` - Server error

POST /api/aa/sponsor-metamask

Sponsors UserOperations for MetaMask smart accounts.

Request: Same as `/api/aa/sponsor`

Response: Same as `/api/aa/sponsor`

AI Resolution

POST /api/ai/resolve

Resolves a prediction market using AI analysis.

Request:

```
{
  "marketId": 1,

  "question": "Did Bitcoin reach $100,000 in 2024?",

  "resolutionDate": "2024-12-31"

}
```

Response (Success):


```
{
  "marketId": 1,

  "outcome": "NO",

  "confidence": 100,

  "reasoning": "According to multiple sources, Bitcoin's price on December 31

  "sources": [

    "https://coinmarketcap.com/...",

    "https://coingecko.com/..."

  ],

  "latency_ms": 1772

}
```


Response (Invalid):


```
{
  "marketId": 1,

  "outcome": "INVALID",
```

```
  "confidence": 0,

  "reasoning": "Question is ambiguous or lacks verifiable data",

  "latency_ms": 1234

}
```

Response (Error):

```
{
  "error": "Failed to resolve market",

  "details": "GROQ_API_KEY not configured"

}
```

Status Codes:

- `200` - Success
- `400` - Missing required fields
- `500` - Server error

Outcome Values:

- `YES` - Market question is true
- `NO` - Market question is false
- `INVALID` - Question is unresolvable

Confidence Range: 0-100

- 90-100: Very high confidence

- 70-89: High confidence
- 50-69: Moderate confidence
- 0-49: Low confidence

Faucet

POST /api/faucet

Distributes testnet tokens to users.

Request:

```
{
  "address": "0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb"

}
```

Response (Success):

```
{
  "success": true,

  "txHash": "0x...",

  "amount": "100000000000000000",

  "token": "BNB"

}
```

Response (Error):

```
{
  "error": "Rate limit exceeded. Try again in 24 hours."

}
```

Status Codes:

- `200` - Success
- `400` - Invalid address
- `429` - Rate limit exceeded
- `500` - Server error

Rate Limits:

- 1 request per address per 24 hours
- 10 requests per IP per hour

POST /api/faucet/busd

Distributes tBUSD tokens to users.

Request:

```
{
  "address": "0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb"

}
```

Response (Success):

```
{
  "success": true,
```

```
  "txHash": "0x...",

  "amount": "3000000000000000000",

  "token": "tBUSD"

}
```

Response (Error):

```
{
  "error": "Already claimed in last 24 hours"

}
```

Status Codes:

- `200` - Success
- `400` - Invalid address
- `429` - Rate limit exceeded
- `500` - Server error

Amount: 30 tBUSD per request

Rate Limits:

- 1 request per address per 24 hours

Oracle Resolution

POST /api/oracle/resolve

Oracle endpoint for market resolution (future UMA integration).

Request:

```
{
  "marketId": 1,

  "outcome": true

}
```

Response:

```
{
  "success": true,

  "marketId": 1,

  "outcome": true

}
```

Status Codes:

- `200` - Success
- `400` - Invalid request
- `500` - Server error

Rate Limits

Global Limits

- 100 requests per IP per hour
- 1000 requests per IP per day

Endpoint-Specific Limits

| Endpoint | Limit |

|----------|-------|

| `/api/aa/sponsor` | 50 per user per hour |

| `/api/ai/resolve` | 10 per user per hour |

| `/api/faucet` | 1 per address per 24h |

| `/api/faucet/busd` | 1 per address per 24h |

Rate Limit Headers

```
X-RateLimit-Limit: 100


X-RateLimit-Remaining: 95


X-RateLimit-Reset: 1640995200
```

Error Codes

400 Bad Request

Invalid request parameters.

Example:

```
{
  "error": "Missing required field: question"


}
```

401 Unauthorized

Authentication required (future).

429 Too Many Requests

Rate limit exceeded.

Example:

```
{
  "error": "Rate limit exceeded. Try again in 3600 seconds."

}
```

500 Internal Server Error

Server-side error.

Example:

```
{
  "error": "Internal server error",

  "details": "Database connection failed"

}
```

Response Format

All API responses follow this structure:

Success:

```
{
  "data": { ... },
```

```
  "latency_ms": 234

}
```

Error:

```
{
  "error": "Error message",

  "details": "Additional details"

}
```

CORS

CORS enabled for all origins in development.

Production: Restricted to `seerhive.vercel.app`

Webhooks (Future)

Subscribe to events:

- Market created
- Market resolved
- Trade executed
- Challenge submitted

Example:

```
{
  "event": "market.resolved",

  "data": {

    "marketId": 1,

    "outcome": true,

    "timestamp": 1640995200

  }

}
```

SDK (Future)

TypeScript SDK for easier integration:

```
import { SeerHiveClient } from '@seerhive/sdk';

const client = new SeerHiveClient({

  apiKey: 'your_api_key'

});

// Resolve market

const result = await client.ai.resolve({
```

```
  marketId: 1,

  question: 'Did Bitcoin reach $100k?',

  resolutionDate: '2024-12-31'

});
// Sponsor transaction

const sponsored = await client.aa.sponsor(userOp);
```

Testing

Test Endpoints

```
# Test paymaster

./scripts/test-sponsor.sh

# Test AI resolution

./scripts/test-ai-resolve.sh

# Test faucet

curl -X POST http://localhost:3000/api/faucet/busd \
```

```
  -H "Content-Type: application/json" \

  -d '{"address": "0x..."}'
```

Postman Collection

Import collection: [Coming Soon]

Next Steps

- Gasless Transactions - ERC-4337 details
- AI Resolution - AI system details
- Testing - Testing guide

Configuration

Complete guide to configuring SeerHive for development and production.

Environment Variables

Frontend Configuration

Location: `apps/web/.env.local`

```
# ============================================================
# MODE
# ============================================================
NEXT_PUBLIC_DEMO=0                    # 0=on-chain, 1=demo mode

# ============================================================
# NETWORK (BNB Testnet)
# ============================================================
NEXT_PUBLIC_CHAIN=bscTestnet
```

```
NEXT_PUBLIC_CHAIN_ID=97

NEXT_PUBLIC_RPC_URL=https://bsc-testnet.publicnode.com

# ============================================================

# SMART CONTRACTS

# ============================================================

NEXT_PUBLIC_CONTRACT_ADDRESS=0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127

NEXT_PUBLIC_ENTRY_POINT=0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789

# ============================================================

# WALLETCONNECT (Optional)

# ============================================================

NEXT_PUBLIC_WC_PROJECT_ID=your_walletconnect_project_id

# ============================================================

# PARTICLE NETWORK (Client-side)
```

```
# ============================================================

NEXT_PUBLIC_PARTICLE_PROJECT_ID=your_project_id

NEXT_PUBLIC_PARTICLE_CLIENT_KEY=your_client_key

NEXT_PUBLIC_PARTICLE_APP_ID=your_app_id

# ============================================================
```

PAYMASTER ROUTING

```
# ============================================================

NEXT_PUBLIC_PAYMASTER_ROUTING=auto    # auto|pimlico|particle|abtest

# ============================================================
```

PARTICLE PAYMASTER (Server-side)

```
# ============================================================

PARTICLE_PAYMASTER_URL=https://paymaster.particle.network/chain/97
```

```
PARTICLE_PROJECT_ID=your_project_id

PARTICLE_CLIENT_KEY=your_client_key

# ============================================================

# PIMLICO PAYMASTER (Server-side)

# ============================================================

PIMLICO_URL=https://api.pimlico.io/v2/97/rpc?apikey=YOUR_API_KEY

NEXT_PUBLIC_PIMLICO_API_KEY=your_api_key

# ============================================================

# AI RESOLUTION (Server-side)

# ============================================================

GROQ_API_KEY=gsk_your_groq_key_here

TAVILY_API_KEY=tvly-your_tavily_key_here

# ============================================================
```

```
# FAUCET (Server-side)
# ============================================================

DEPLOYER_PRIVATE_KEY=your_deployer_private_key

FAUCET_PRIVATE_KEY=your_faucet_private_key
```

Contract Configuration

Location: `contracts/.env`

```
# ============================================================
# DEPLOYMENT
# ============================================================

PRIVATE_KEY=your_deployer_private_key

BSC_TESTNET_RPC=https://bsc-testnet.publicnode.com
```

```
# ============================================================

# VERIFICATION (Optional)

# ============================================================

BSCSCAN_API_KEY=your_bscscan_api_key
```

Configuration Sections

1. Mode Configuration

Demo Mode

Test all features without wallet connection:

```
NEXT_PUBLIC_DEMO=1
```

Features:

- Mock data for markets, trades, analytics
- No wallet connection required
- No blockchain interactions
- Perfect for UI/UX testing

  #### On-Chain Mode

  Real blockchain interactions:

  ```
  NEXT_PUBLIC_DEMO=0
  ```

Features:

- Real wallet connection (MetaMask, Particle)
- Live contract interactions
- Gasless transactions
- Requires testnet tokens

2. Network Configuration

BNB Testnet (Default)

```
NEXT_PUBLIC_CHAIN=bscTestnet

NEXT_PUBLIC_CHAIN_ID=97

NEXT_PUBLIC_RPC_URL=https://bsc-testnet.publicnode.com
```

BNB Mainnet (Future)

```
NEXT_PUBLIC_CHAIN=bsc

NEXT_PUBLIC_CHAIN_ID=56

NEXT_PUBLIC_RPC_URL=https://bsc-dataseed.binance.org
```

3. Smart Contract Configuration

Deployed Contracts

```
# PredictionMarket contract


NEXT_PUBLIC_CONTRACT_ADDRESS=0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127

# ERC-4337 EntryPoint


NEXT_PUBLIC_ENTRY_POINT=0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789
```

Custom Deployment

If you deploy your own contracts:

```
cd contracts


pnpm deploy:testnet


# Copy new contract address to .env.local
```

4. Wallet Configuration

WalletConnect

Get project ID: cloud.walletconnect.com

```
NEXT_PUBLIC_WC_PROJECT_ID=your_project_id
```

Particle Network

Get credentials: dashboard.particle.network

Client-side (social login):

```
NEXT_PUBLIC_PARTICLE_PROJECT_ID=your_project_id

NEXT_PUBLIC_PARTICLE_CLIENT_KEY=your_client_key


NEXT_PUBLIC_PARTICLE_APP_ID=your_app_id
```

Server-side (paymaster):

```
PARTICLE_PROJECT_ID=your_project_id

PARTICLE_CLIENT_KEY=your_client_key
```

5. Paymaster Configuration

Routing Strategy

```
NEXT_PUBLIC_PAYMASTER_ROUTING=auto
```

Options:

- `auto` - Try Particle first, fallback to Pimlico (recommended)
- `particle` - Only use Particle
- `pimlico` - Only use Pimlico
- `abtest` - A/B test between providers

Particle Paymaster

Get credentials: dashboard.particle.network

```
PARTICLE_PAYMASTER_URL=https://paymaster.particle.network/chain/97
```

```
PARTICLE_PROJECT_ID=your_project_id


PARTICLE_CLIENT_KEY=your_client_key
```

Pimlico Paymaster

Get API key: dashboard.pimlico.io

```
PIMLICO_URL=https://api.pimlico.io/v2/97/rpc?apikey=YOUR_API_KEY


NEXT_PUBLIC_PIMLICO_API_KEY=your_api_key
```

6. AI Configuration

Groq (LLM)

Get free API key: console.groq.com

```
GROQ_API_KEY=gsk_your_groq_key_here
```

Rate Limits:

- Free tier: 30 requests/minute
- Models: Llama 3.3 70B, Llama 3.1 8B

Tavily (Web Search)

Get free API key: tavily.com

```
TAVILY_API_KEY=tvly-your_tavily_key_here
```

Rate Limits:

- Free tier: 1000 searches/month

7. Faucet Configuration

Auto-Fund Smart Accounts

```
DEPLOYER_PRIVATE_KEY=your_deployer_private_key
```

Used to automatically fund new smart accounts with testnet tokens.

tBUSD Faucet

```
FAUCET_PRIVATE_KEY=your_faucet_private_key
```

Private key for tBUSD faucet contract.

Security Best Practices

Client-Side vs Server-Side

✓ Client-Side (NEXT_PUBLIC_ prefix)

Safe to expose in browser:

```
NEXT_PUBLIC_CHAIN_ID=97
NEXT_PUBLIC_CONTRACT_ADDRESS=0x...

NEXT_PUBLIC_PARTICLE_PROJECT_ID=xxx
```

× Server-Side (No prefix)

Never expose in browser:

```
GROQ_API_KEY=gsk_...
TAVILY_API_KEY=tvly-...
```

```
PARTICLE_CLIENT_KEY=xxx

DEPLOYER_PRIVATE_KEY=0x...
```

Private Key Management

Never commit private keys to Git:

```
# .gitignore

.env

.env.local

.env.production
```

Use separate keys for testnet and mainnet:

```
# Testnet

PRIVATE_KEY=0x...testnet_key...

# Mainnet
```

```
PRIVATE_KEY=0x...mainnet_key...
```

API Key Rotation

Rotate keys regularly:

- Groq: Every 90 days
- Tavily: Every 90 days
- Particle: Every 180 days
- Pimlico: Every 180 days

Configuration Files

TypeScript Config

`apps/web/tsconfig.json`:

```
{
  "extends": "@repo/config/tsconfig.json",

  "compilerOptions": {

    "target": "ES2022",

    "lib": ["ES2022", "DOM"],

    "strict": true,

    "paths": {

      "@/*": ["./src/*"]

    }
```

```
  }


}
```

Next.js Config

`apps/web/next.config.js`:

```
module.exports = {
  reactStrictMode: true,

  transpilePackages: ["@repo/config"],

  webpack: (config) => {

    config.resolve.fallback = { fs: false, net: false, tls: false };

    return config;

  }

};
```

Tailwind Config

`apps/web/tailwind.config.ts`:

```
import type { Config } from 'tailwindcss';
const config: Config = {
  content: ['./src/*/.{js,ts,jsx,tsx,mdx}'],
```

```
  theme: {

    extend: {

      colors: {

        slate: { / ... / }

      }

    }

  },

  plugins: []

};
export default config;
```

Hardhat Config

`contracts/hardhat.config.ts`:

```
import { HardhatUserConfig } from 'hardhat/config';
import '@nomicfoundation/hardhat-toolbox';

import * as dotenv from 'dotenv';
dotenv.config();
const config: HardhatUserConfig = {
```

```
  solidity: {
    version: '0.8.20',
    settings: {
      optimizer: {
        enabled: true,
        runs: 200
      }
    }
  },
  networks: {
    bscTestnet: {
      url: process.env.BSC_TESTNET_RPC,
      accounts: [process.env.PRIVATE_KEY!],
      chainId: 97
    }
  }
```

```
};

export default config;
```

Troubleshooting

"Missing environment variable"

Cause: Required variable not set

Solution: Copy from `.env.example` and fill in values

"Invalid API key"

Cause: Incorrect or expired API key

Solution: Generate new key from provider dashboard

"Network mismatch"

Cause: Wallet on different network than configured

Solution: Switch wallet to BNB Testnet (Chain ID: 97)

"Contract not found"

Cause: Wrong contract address or network

Solution: Verify contract address matches deployed contract

Environment Templates

Minimal (Demo Mode)

```
NEXT_PUBLIC_DEMO=1
```

Development (On-Chain)

```
NEXT_PUBLIC_DEMO=0
```

```
NEXT_PUBLIC_CHAIN_ID=97

NEXT_PUBLIC_CONTRACT_ADDRESS=0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127

NEXT_PUBLIC_RPC_URL=https://bsc-testnet.publicnode.com
```

Full (All Features)

```
# Mode

NEXT_PUBLIC_DEMO=0

# Network

NEXT_PUBLIC_CHAIN_ID=97

NEXT_PUBLIC_CONTRACT_ADDRESS=0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127

# Paymasters

PARTICLE_PROJECT_ID=xxx

PARTICLE_CLIENT_KEY=xxx

PIMLICO_URL=https://api.pimlico.io/v2/97/rpc?apikey=xxx

# AI
```

```
GROQ_API_KEY=gsk_xxx

TAVILY_API_KEY=tvly-xxx
```

Next Steps

- Getting Started - Setup guide
- Testing - Test configuration
- Deployment - Production deployment

Testing

Comprehensive testing guide for SeerHive.

Testing Strategy

SeerHive uses multiple testing approaches:

1. **Smart Contract Tests** - Hardhat + Chai
2. **API Tests** - Shell scripts + curl
3. **Integration Tests** - End-to-end flows
4. **Manual Testing** - UI/UX validation

Smart Contract Testing

Setup

```
cd contracts


pnpm install
```

Run Tests

```
pnpm test
```

Test Structure

Location: `contracts/test/PredictionMarket.test.ts`

```
import { expect } from 'chai';


import { ethers } from 'hardhat';

describe('PredictionMarket', function () {
```

```
  let market: PredictionMarket;

  let owner: SignerWithAddress;

  let user1: SignerWithAddress;
  beforeEach(async function () {

    [owner, user1] = await ethers.getSigners();

    const PredictionMarket = await ethers.getContractFactory('PredictionMarket');

    market = await PredictionMarket.deploy();

  });
  it('Should create a market', async function () {

    const tx = await market.createMarket('Test question?', 86400);

    await tx.wait();




    const marketData = await market.markets(0);

    expect(marketData.question).to.equal('Test question?');

  });

});
```

Test Coverage

Run with coverage:

```
pnpm test:coverage
```

Current Coverage:

- Market creation:
- Share purchases (BNB):
- Share purchases (ERC20):
- Resolution proposal:
- Challenge mechanism:
- Finalization:
- Winnings claims:

Key Test Cases

1. Market Creation

```
it('Should create a market', async function () {

  const tx = await market.createMarket('Will BNB reach $1000?', 86400);

  await tx.wait();



  const marketData = await market.markets(0);

  expect(marketData.question).to.equal('Will BNB reach $1000?');
```

```
  expect(marketData.resolved).to.equal(false);


});
```

2. Buy Shares (BNB)

```
it('Should allow buying YES shares with BNB', async function () {

  await market.createMarket('Test?', 86400);

  const amount = parseEther('1');




  await market.buyShares(0, true, ethers.ZeroAddress, amount, {

    value: amount

  });




  const shares = await market.userShares(0, user1.address, true);

  expect(shares).to.equal(amount);

});
```

3. Buy Shares (ERC20)

```
it('Should allow buying shares with ERC20', async function () {

  const MockERC20 = await ethers.getContractFactory('MockERC20');

  const token = await MockERC20.deploy('Test', 'TST');



  await token.mint(user1.address, parseEther('100'));

  await token.connect(user1).approve(market.address, parseEther('10'));



  await market.connect(user1).buyShares(0, true, token.address, parseEther('1

  const shares = await market.userShares(0, user1.address, true);

  expect(shares).to.equal(parseEther('10'));

});
```

4. Resolution & Challenge

```
it('Should allow proposing and challenging resolution', async function () {
```

```
  await market.createMarket('Test?', 1); // 1 second duration

  await new Promise(resolve => setTimeout(resolve, 2000));



  // Propose resolution

  await market.proposeResolution(0, true);



  // Challenge

  await market.connect(user1).challengeResolution(0, {

    value: parseEther('0.01')

  });



  const resolution = await market.resolutions(0);

  expect(resolution.challenged).to.equal(true);

});
```

5. Claim Winnings

```
it('Should allow claiming winnings', async function () {

  await market.createMarket('Test?', 1);

  await market.buyShares(0, true, ethers.ZeroAddress, parseEther('1'), {

    value: parseEther('1')

  });




  await new Promise(resolve => setTimeout(resolve, 2000));

  await market.finalizeMarket(0, true);




  const balanceBefore = await ethers.provider.getBalance(user1.address);

  await market.connect(user1).claimWinnings(0);

  const balanceAfter = await ethers.provider.getBalance(user1.address);




  expect(balanceAfter).to.be.gt(balanceBefore);

});
```

API Testing

Paymaster API Test

Test gasless transaction sponsorship:

```
./scripts/test-sponsor.sh
```

Script:

```
#!/bin/bash

curl -X POST http://localhost:3000/api/aa/sponsor \

  -H "Content-Type: application/json" \


  -d '{


    "userOp": {


      "sender": "0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb",


      "nonce": "0x0",


      "initCode": "0x",


      "callData": "0x",


      "callGasLimit": "0x0",


      "verificationGasLimit": "0x0",


      "preVerificationGas": "0x0",
```

```
      "maxFeePerGas": "0x0",

      "maxPriorityFeePerGas": "0x0",

      "paymasterAndData": "0x",

      "signature": "0x"

    },

    "entryPoint": "0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789",

    "chainId": 97

  }'
```

Expected Output:

```
{
  "paymasterAndData": "0x...",

  "provider": "particle",

  "latency_ms": 234

}
```

AI Resolution Test

Test AI-powered market resolution:

```
./scripts/test-ai-resolve.sh
```

Script:

```
#!/bin/bash

curl -X POST http://localhost:3000/api/ai/resolve \

  -H "Content-Type: application/json" \


  -d '{


    "marketId": 999,


    "question": "Did Bitcoin reach $100,000 in 2024?",


    "resolutionDate": "2024-12-31"


  }'
```

Expected Output:

```
{

  "marketId": 999,


  "outcome": "NO",


  "confidence": 100,
```

```
  "reasoning": "Historical data shows Bitcoin did not reach $100,000 in 2024.

  "latency_ms": 1772

}
```

Faucet Test

Test tBUSD faucet:

```
curl -X POST http://localhost:3000/api/faucet/busd \

  -H "Content-Type: application/json" \

  -d '{

    "address": "0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb"

  }'
```

Expected Output:

```
{
  "success": true,

  "txHash": "0x...",

  "amount": "3000000000000000000",
```

```
  "token": "tBUSD"

}
```

Integration Testing

Gasless Transaction Flow

Test complete gasless transaction:

```
./scripts/smoke-gasless.sh
```

Flow:

1. Construct UserOperation
2. Request sponsorship from `/api/aa/sponsor`
3. Submit to bundler
4. Verify on-chain

Expected Output:

```
 UserOperation constructed
 Sponsorship received: particle

 Transaction submitted: 0x...

 Transaction confirmed

 User paid 0 gas
```

Market Creation Flow

Manual Test:

1. Start dev server: `pnpm dev`

2. Connect wallet (MetaMask)

3. Go to `/markets`

4. Click "Create Market"

5. Fill form:

- Question: "Will BNB reach $1000 by 2025?"

- Duration: 365 days

6. Toggle "Use Gasless Transaction"

7. Submit

8. Verify on BSCScan

Trading Flow

Manual Test:

1. Go to `/markets`

2. Select a market

3. Click "Trade"

4. Select token (tBUSD)

5. Enter amount (10)

6. Choose side (YES)

7. Toggle "Use Gasless Transaction"

8. Submit

9. Verify shares in portfolio

Resolution Flow

Manual Test:

1. Create market with 1-minute duration

2. Wait for expiry

3. Click "Resolve with AI"

4. Review AI decision

5. Submit resolution proposal

6. Wait 24 hours (or test challenge)

7. Finalize market

8. Claim winnings

Demo Mode Testing

Test all features without wallet:

```
# Set demo mode

echo "NEXT_PUBLIC_DEMO=1" > apps/web/.env.local

# Start server

pnpm dev

# Visit http://localhost:3000
```

Test Cases:

- Browse markets
- View analytics
- Create market (mock)
- Trade shares (mock)
- View portfolio
- Check transaction history

Performance Testing

API Latency

Measure API response times:

```
# Paymaster API

time curl -X POST http://localhost:3000/api/aa/sponsor -d '{...}'

# AI Resolution API

time curl -X POST http://localhost:3000/api/ai/resolve -d '{...}'
```

Targets:

- Paymaster: < 500ms
- AI Resolution: < 3000ms

Load Testing

Use `ab` (Apache Bench):

```
# Test paymaster endpoint

ab -n 100 -c 10 -p payload.json -T application/json \

  http://localhost:3000/api/aa/sponsor
```

Targets:

- 100 requests/second
- < 1% error rate

Test Data

Mock Markets

Location: `apps/web/src/mocks/fixtures/`

markets.json:

```
[
  {
    "id": 1,
    "question": "Will BNB reach $1000 by end of 2025?",
    "totalYesShares": "10000000000000000000",
    "totalNoShares": "50000000000000000000",
    "endTime": 1735689600,
    "resolved": false
  }
]
```

Test Tokens

BNB Testnet Faucet: testnet.bnbchain.org/faucet-smart

tBUSD Faucet: localhost:3000/faucet

Test Accounts

Create test accounts:

Generate new wallet

```
npx hardhat run scripts/generate-wallet.ts
```

Fund with testnet tokens

Use faucets above

```
```

Continuous Integration

GitHub Actions

Location: `.github/workflows/ci.yml`

```
name: CI

on: [push, pull_request]

jobs:

  test:

    runs-on: ubuntu-latest

    steps:

      - uses: actions/checkout@v3

      - uses: pnpm/action-setup@v2
```

```
      - uses: actions/setup-node@v3

        with:

          node-version: 20

          cache: 'pnpm'




      - run: pnpm install

      - run: pnpm lint

      - run: pnpm test

      - run: pnpm build
```

Pre-commit Hooks

Install Husky:

```
pnpm add -D husky


npx husky install
```

Add pre-commit hook:

```
#!/bin/sh
```

```
pnpm lint


pnpm test
```

Debugging

Contract Debugging

Use Hardhat console:

```
cd contracts


npx hardhat console --network bscTestnet
```

```
const market = await ethers.getContractAt('PredictionMarket', '0x...');


const data = await market.markets(0);


console.log(data);
```

Frontend Debugging

Enable debug logs:

```
DEBUG=* pnpm dev
```

Check console for:

- ✓ Success indicators
- ⚠ Warnings
- ✗ Errors

Network Debugging

Monitor transactions:

```
# Watch for new blocks

npx hardhat run scripts/watch-blocks.ts --network bscTestnet

# Monitor specific address

npx hardhat run scripts/watch-address.ts --network bscTestnet
```

Test Checklist

Before Deployment

- [ ] All contract tests pass
- [ ] API tests pass
- [ ] Gasless flow works
- [ ] AI resolution works
- [ ] Demo mode works
- [ ] Manual testing complete
- [ ] No console errors
- [ ] Performance targets met

After Deployment

- [ ] Contract verified on BSCScan
- [ ] Faucet working
- [ ] Paymaster funded
- [ ] AI APIs configured
- [ ] Monitoring enabled
- [ ] Error tracking setup

Troubleshooting

Tests Failing

Check:

1. Node version (20.x required)
2. Dependencies installed (`pnpm install`)
3. Environment variables set
4. Network connection

Gasless Tests Failing

Check:

1. Paymaster API keys valid
2. Paymaster accounts funded
3. EntryPoint address correct
4. Network is BNB Testnet

AI Tests Failing

Check:

1. Groq API key valid
2. Tavily API key valid
3. Rate limits not exceeded
4. Network connection

Next Steps

- Deployment - Deploy to production
- Configuration - Environment setup
- API Reference - API documentation

Deployment

Guide to deploying SeerHive to production.

Overview

SeerHive consists of two main components:

1. **Smart Contracts** - Deployed to BNB Chain

2. **Frontend Application** - Deployed to Vercel

Prerequisites

- Node.js 20.x or higher
- pnpm 10.x or higher
- BNB for gas fees (testnet or mainnet)
- Vercel account (for frontend)
- API keys (Particle, Pimlico, Groq, Tavily)

Smart Contract Deployment

1. Prepare Environment

```
cd contracts


cp .env.example .env
```

Edit `contracts/.env`:

```
PRIVATE_KEY=your_deployer_private_key


BSC_TESTNET_RPC=https://bsc-testnet.publicnode.com


BSCSCAN_API_KEY=your_bscscan_api_key
```

2. Compile Contracts

```
pnpm build
```

Verify compilation:

```
ls artifacts/contracts/PredictionMarket.sol/
```

Should see PredictionMarket.json

3. Deploy to BNB Testnet

```
pnpm deploy:testnet
```

Expected Output:

```
Deploying PredictionMarket...

PredictionMarket deployed to: 0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127
```

Save the contract address - you'll need it for frontend configuration.

4. Verify Contract

```
npx hardhat verify --network bscTestnet 0xYourContractAddress
```

Expected Output:

```
Successfully verified contract PredictionMarket on BSCScan

https://testnet.bscscan.com/address/0x...#code
```

5. Deploy to BNB Mainnet

⚠ **Only after thorough testing and audit**

Update `contracts/.env`:

```
PRIVATE_KEY=your_mainnet_deployer_key

BSC_MAINNET_RPC=https://bsc-dataseed.binance.org
```

Update `hardhat.config.ts`:

```
networks: {

  bscMainnet: {

    url: process.env.BSC_MAINNET_RPC,

    accounts: [process.env.PRIVATE_KEY!],

    chainId: 56

  }

}
```

Deploy:

```
npx hardhat run scripts/deploy.ts --network bscMainnet
```

Frontend Deployment

1. Prepare Environment

```
cd apps/web

cp .env.example .env.production
```

Edit `apps/web/.env.production`:

```
# Mode

NEXT_PUBLIC_DEMO=0

# Network (Testnet)

NEXT_PUBLIC_CHAIN=bscTestnet

NEXT_PUBLIC_CHAIN_ID=97

NEXT_PUBLIC_RPC_URL=https://bsc-testnet.publicnode.com

# Or Mainnet
```

NEXT_PUBLIC_CHAIN=bsc

NEXT_PUBLIC_CHAIN_ID=56

NEXT_PUBLIC_RPC_URL=https://bsc-datasee

Contract (use your deployed address)

NEXT_PUBLIC_CONTRACT_ADDRESS=0xYourDeployedContractAddress

NEXT_PUBLIC_ENTRY_POINT=0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789

Particle Network

NEXT_PUBLIC_PARTICLE_PROJECT_ID=your_project_id

NEXT_PUBLIC_PARTICLE_CLIENT_KEY=your_client_key

NEXT_PUBLIC_PARTICLE_APP_ID=your_app_id

Paymasters (Server-side)

PARTICLE_PAYMASTER_URL=https://paymaster.particle.network/chain/97

PARTICLE_PROJECT_ID=your_project_id

```
PARTICLE_CLIENT_KEY=your_client_key


PIMLICO_URL=https://api.pimlico.io/v2/97/rpc?apikey=YOUR_KEY

# AI (Server-side)


GROQ_API_KEY=gsk_your_groq_key


TAVILY_API_KEY=tvly-your_tavily_key

# Faucet (Server-side)


DEPLOYER_PRIVATE_KEY=your_deployer_key


FAUCET_PRIVATE_KEY=your_faucet_key
```

2. Build Application

```
cd ../..  # Back to root


pnpm build
```

Verify build:

```
ls apps/web/.next/

# Should see build artifacts
```

3. Deploy to Vercel

Option A: Vercel CLI

Install Vercel CLI:

```
npm i -g vercel
```

Deploy:

```
cd apps/web

vercel
```

Follow prompts:

- Link to existing project or create new
- Set environment variables
- Deploy

Option B: GitHub Integration

1. Push code to GitHub

2. Go to vercel.com

3. Click "New Project"

4. Import your GitHub repository

5. Configure:

- Framework: Next.js

- Root Directory: `apps/web`

- Build Command: `cd ../.. && pnpm build --filter=web`

- Output Directory: `.next`

6. Add environment variables (from `.env.production`)

7. Deploy

4. Configure Environment Variables

In Vercel dashboard:

Settings → Environment Variables

Add all variables from `.env.production`:

| Key | Value | Environment |

|-----|-------|-------------|

| NEXT_PUBLIC_DEMO | 0 | Production |

| NEXT_PUBLIC_CHAIN_ID | 97 | Production |

| NEXT_PUBLIC_CONTRACT_ADDRESS | 0x... | Production |

| GROQ_API_KEY | gsk_... | Production |

| TAVILY_API_KEY | tvly-... | Production |

| ... | ... | ... |

⚠ **Important**: Server-side variables (without `NEXT_PUBLIC_`) are encrypted.

5. Configure Custom Domain

In Vercel dashboard:

Settings → Domains

Add your domain:

- `seerhive.com`
- `www.seerhive.com`

Update DNS records:

```
Type: CNAME

Name: @


Value: cname.vercel-dns.com
```

6. Verify Deployment

Visit your deployed URL:

```
https://seerhive.vercel.app
```

Test:

- Homepage loads
- Markets page works
- Wallet connection works
- Gasless transactions work
- AI resolution works

Post-Deployment

1. Fund Paymaster Accounts

Ensure paymasters have sufficient funds:

Particle Network:

- Check balance in dashboard.particle.network
- Top up if needed

Pimlico:

- Check balance in dashboard.pimlico.io
- Top up if needed

2. Monitor Application

Set up monitoring:

Vercel Analytics:

- Enable in Vercel dashboard
- Track page views, performance

Error Tracking (Sentry):

```
pnpm add @sentry/nextjs
```

Configure `sentry.config.js`:

```
Sentry.init({
  dsn: 'your_sentry_dsn',

  environment: 'production'
```

```
});
```

Uptime Monitoring:

- Use UptimeRobot
- Monitor `/api/health` endpoint

3. Set Up Alerts

Configure alerts for:

- Paymaster balance low
- API rate limits exceeded
- Contract errors
- High error rates

4. Enable Rate Limiting

Add rate limiting middleware:

`apps/web/src/middleware.ts`:

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
const rateLimit = new Map();
export function middleware(request: NextRequest) {

  const ip = request.ip || 'unknown';

  const now = Date.now();

  const windowMs = 60  1000; // 1 minute

  const max = 100; // 100 requests per minute
  if (!rateLimit.has(ip)) {
```

```
    rateLimit.set(ip, []);

  }
  const requests = rateLimit.get(ip).filter((time: number) => now - time < wi



  if (requests.length >= max) {

    return NextResponse.json(

      { error: 'Too many requests' },

      { status: 429 }

    );

  }
  requests.push(now);

  rateLimit.set(ip, requests);
  return NextResponse.next();

}
export const config = {

  matcher: '/api/:path'

};
```

Rollback Procedure

If deployment fails:

Vercel Rollback

1. Go to Vercel dashboard

2. Click "Deployments"

3. Find previous working deployment

4. Click "..." → "Promote to Production"

Contract Rollback

⚠ **Contracts are immutable - cannot rollback**

Options:

1. Deploy new contract with fixes

2. Update frontend to use old contract

3. Use proxy pattern (future)

Maintenance

Update Dependencies

```
pnpm update


pnpm audit


pnpm audit fix
```

Update Contracts

1. Make changes to contracts

2. Test thoroughly

3. Deploy new contract

4. Update frontend contract address

5. Redeploy frontend

Update Frontend

```
git pull

pnpm install

pnpm build

vercel --prod
```

Security Checklist

Before Mainnet Deployment

- [ ] Smart contract audit completed
- [ ] All tests passing
- [ ] Security review done
- [ ] Rate limiting enabled
- [ ] Error tracking configured
- [ ] Monitoring set up
- [ ] Backup plan ready
- [ ] Team trained on incident response

Environment Security

- [ ] Private keys stored securely (not in code)
- [ ] API keys rotated regularly
- [ ] Server-side variables not exposed
- [ ] HTTPS enabled
- [ ] CORS configured properly
- [ ] Rate limiting active

Cost Estimates

Monthly Costs (Testnet)

| Service | Cost |

|---------|------|

| Vercel (Hobby) | $0 |

| Particle Paymaster | ~$100 |

| Pimlico Paymaster | ~$50 |

| Groq API | $0 (free tier) |

| Tavily API | $0 (free tier) |

| **Total** | **~$150** |

Monthly Costs (Mainnet)

| Service | Cost |

|---------|------|

| Vercel (Pro) | $20 |

| Particle Paymaster | ~$500 |

| Pimlico Paymaster | ~$300 |

| Groq API | ~$50 |

| Tavily API | ~$50 |

| Monitoring | ~$30 |

| **Total** | **~$950** |

Scaling Considerations

Horizontal Scaling

Vercel automatically scales:

- Serverless functions
- Edge network
- CDN caching

Database (Future)

When adding database:

- Use Vercel Postgres

- Or external (Supabase, PlanetScale)
- Enable connection pooling

Caching

Implement caching:

- Redis for API responses
- CDN for static assets
- Browser caching headers

Troubleshooting

Deployment Fails

Check:

1. Build logs in Vercel
2. Environment variables set
3. Dependencies installed
4. Node version correct

Contract Not Found

Check:

1. Contract address correct
2. Network matches (testnet vs mainnet)
3. Contract verified on BSCScan

Gasless Transactions Failing

Check:

1. Paymaster accounts funded
2. API keys valid
3. Rate limits not exceeded

AI Resolution Failing

Check:

1. Groq API key valid
2. Tavily API key valid
3. Rate limits not exceeded

Next Steps

- Configuration - Environment setup
- Testing - Testing guide
- API Reference - API documentation

Contributing

Thank you for your interest in contributing to SeerHive! This guide will help you get started.

Ways to Contribute

- Report bugs
- Suggest features
- Improve documentation
- Fix issues
- Add new features
- Write tests
- Improve UI/UX

Getting Started

1. Fork Repository

```
# Fork on GitHub, then clone

git clone https://github.com/YOUR_USERNAME/seerhive.git

cd seerhive
```

2. Install Dependencies

```
pnpm install
```

3. Create Branch

```
git checkout -b feature/your-feature-name
```

or

```
git checkout -b fix/your-bug-fix
```

4. Make Changes

Follow our Development Guidelines

5. Test Changes

```
# Run tests
pnpm test

# Run linter
pnpm lint

# Build project
pnpm build
```

6. Commit Changes

```
git add .

git commit -m "feat: add new feature"

or

git commit -m "fix: resolve bug"
```

Follow Conventional Commits

7. Push & Create PR

```
git push origin feature/your-feature-name
```

Create Pull Request on GitHub with:

- Clear description
- Related issue number
- Screenshots (if UI changes)
- Test results

Development Guidelines

Code Style

Follow existing patterns in the codebase:

TypeScript:

- Strict mode enabled
- Explicit types preferred
- Avoid `any` except in error handling

React:

- Functional components
- Hooks for state management
- Component variants for different providers

Naming:

- Components: PascalCase (`TradeDialog.tsx`)
- Utilities: camelCase (`gasless.ts`)
- Constants: UPPER_SNAKE_CASE (`PREDICTION_MARKET_ABI`)

File Organization

```
// 1. Imports (external, internal, types, assets)

import { useState } from 'react';

import { useStore } from '@/lib/store';

import type { Market } from '@/types/market';

import { X } from 'lucide-react';
// 2. Interface definitions

interface Props {

  market: Market;

  onClose: () => void;

}
// 3. Component function

export function Component({ market, onClose }: Props) {
```

```
  // 4. State

  const [loading, setLoading] = useState(false);




  // 5. Hooks

  useEffect(() => { }, []);




  // 6. Handlers

  const handleAction = () => { };




  // 7. JSX

  return
...
;


}
```

Testing Requirements

All contributions should include tests:

Smart Contracts:

```
it('Should create a market', async function () {
  const tx = await market.createMarket('Test?', 86400);

  await tx.wait();




  const data = await market.markets(0);

  expect(data.question).to.equal('Test?');

});
```

API Routes:

```
curl -X POST http://localhost:3000/api/your-endpoint \
  -H "Content-Type: application/json" \

  -d '{"test": "data"}'
```

Frontend:

- Manual testing in demo mode
- Manual testing in on-chain mode
- Screenshot of changes

Documentation

Update documentation for:

- New features
- API changes

- Configuration changes
- Breaking changes

Commit Conventions

Use Conventional Commits:

```
():

[optional body]

[optional footer]


```

Types

- `feat`: New feature
- `fix`: Bug fix
- `docs`: Documentation changes
- `style`: Code style changes (formatting)
- `refactor`: Code refactoring
- `test`: Adding/updating tests
- `chore`: Maintenance tasks

Examples

```
feat(gasless): add Pimlico paymaster fallback


fix(ai): handle Tavily API timeout


docs(wiki): add deployment guide


style(ui): improve button spacing


refactor(contracts): optimize gas usage


```

```
test(api): add sponsor endpoint tests


chore(deps): update dependencies
```

Scope

- `gasless`: Account abstraction
- `ai`: AI resolution
- `contracts`: Smart contracts
- `ui`: User interface
- `api`: API routes
- `docs`: Documentation
- `config`: Configuration

Pull Request Guidelines

PR Title

Use conventional commit format:

```
feat(gasless): add Pimlico paymaster fallback
```

PR Description

Include:

```
## Description


Brief description of changes

## Related Issue
```

```
Fixes #123

## Changes

- Added Pimlico paymaster adapter
- Updated sponsor API to try Particle first
- Added fallback logic

## Testing

- [x] Contract tests pass
- [x] API tests pass
- [x] Manual testing complete

## Screenshots

(if applicable)
```

PR Checklist

- [ ] Code follows style guidelines
- [ ] Tests added/updated
- [ ] Documentation updated
- [ ] Commit messages follow conventions
- [ ] No console errors
- [ ] Builds successfully

Code Review Process

1. **Automated Checks**: CI runs tests and linting
2. **Maintainer Review**: Core team reviews code
3. **Feedback**: Address review comments
4. **Approval**: PR approved by maintainer
5. **Merge**: Squash and merge to main

Issue Guidelines

Bug Reports

Use bug report template:

```
**Describe the bug**

Clear description of the bug

**To Reproduce**

1. Go to '...'

2. Click on '...'

3. See error

**Expected behavior**

What should happen

**Screenshots**

If applicable

**Environment**

- OS: [e.g. macOS]

- Browser: [e.g. Chrome]
```

```
- Version: [e.g. 1.0.0]

**Additional context**


Any other information
```

Feature Requests

Use feature request template:

```
**Is your feature request related to a problem?**


Description of problem

**Describe the solution**


How should it work?

**Describe alternatives**


Other solutions considered

**Additional context**


Any other information
```

Development Setup

Prerequisites

- Node.js 20.x
- pnpm 10.x
- Git

Environment Setup

```
# Frontend

cp apps/web/.env.example apps/web/.env.local

# Edit with your API keys

# Contracts

cp contracts/.env.example contracts/.env

# Add your private key


```

Running Locally

```
# Start all workspaces

pnpm dev

# Or individually

cd apps/web && pnpm dev
```

```
cd contracts && pnpm test
```

Testing

```
# All tests

pnpm test

# Contract tests

cd contracts && pnpm test

# API tests

./scripts/test-sponsor.sh

./scripts/test-ai-resolve.sh

# Gasless flow

./scripts/smoke-gasless.sh
```

Areas for Contribution

High Priority

- [ ] UMA Optimistic Oracle integration
- [ ] Challenge/dispute UI
- [ ] Mobile responsive improvements
- [ ] Performance optimization
- [ ] Security audit fixes

Medium Priority

- [ ] Copy trading features
- [ ] Reputation system
- [ ] Advanced analytics
- [ ] Multi-language support
- [ ] Dark mode improvements

Low Priority

- [ ] Additional chart types
- [ ] Export data features
- [ ] Keyboard shortcuts
- [ ] Accessibility improvements
- [ ] Animation polish

Good First Issues

Look for issues labeled `good first issue`:

- Documentation improvements
- UI polish
- Test coverage
- Bug fixes

Community

Communication

- **GitHub Issues**: Bug reports and feature requests
- **GitHub Discussions**: General questions and ideas
- **Discord**: Coming soon
- **Twitter**: Coming soon

Code of Conduct

Be respectful and inclusive:

- Use welcoming language
- Respect differing viewpoints
- Accept constructive criticism
- Focus on what's best for community

Recognition

Contributors will be:

- Listed in CONTRIBUTORS.md
- Mentioned in release notes
- Credited in documentation

Questions?

- Read Documentation
- Check FAQ
- Open GitHub Discussion
- Contact maintainers

License

By contributing, you agree that your contributions will be licensed under the MIT License.

Next Steps

- Getting Started - Setup development environment
- Architecture - Understand the codebase
- Testing - Learn testing practices

FAQ

Frequently asked questions about SeerHive.

General Questions

What is SeerHive?

SeerHive is a decentralized prediction market platform on BNB Chain where users can bet on real-world events with AI-powered resolution and zero gas fees.

How does it work?

1. Create or browse prediction markets

2. Buy YES or NO shares based on your prediction

3. AI automatically resolves markets using real-time web data

4. Winners claim proportional payouts from the pool

Is it free to use?

Yes! SeerHive uses ERC-4337 account abstraction to sponsor gas fees for users. You only need tokens to trade (BNB, tBUSD, etc.).

What makes SeerHive different?

- **AI Resolution**: Markets resolve in seconds vs 24-48h manual process
- **Zero Gas Fees**: Trade without paying transaction costs
- **Real-time Data**: Web scraping provides current event information
- **BNB Chain Native**: Lower costs and faster finality

Getting Started

How do I start using SeerHive?

1. Visit the platform

2. Connect wallet (MetaMask, Particle Network)

3. Get testnet tokens from faucet

4. Start trading on markets

Do I need a crypto wallet?

For on-chain mode, yes. But you can:

- Use MetaMask (traditional wallet)

- Use Particle Network (social login with Google/Twitter)
- Try demo mode (no wallet required)

Where do I get testnet tokens?

- **BNB**: BNB Testnet Faucet
- **tBUSD**: SeerHive Faucet - 30 tBUSD every 24h
- **Other tokens**: BNB Testnet Faucet

What is demo mode?

Demo mode lets you test all features without wallet connection or blockchain interactions. Perfect for learning the platform.

Enable: `NEXT_PUBLIC_DEMO=1`

Trading

What tokens can I trade with?

BNB Testnet:

- BNB (native token)
- tBUSD (testnet BUSD)
- tUSDT (testnet USDT)
- tUSDC (testnet USDC)
- tDAI (testnet DAI)
- tBTC (testnet BTC)

How do gasless transactions work?

SeerHive sponsors your transactions using ERC-4337 account abstraction. The platform pays gas fees so you don't have to.

Note: Gasless works for all ERC20 tokens. BNB requires small gas fee (~$0.001).

How are payouts calculated?

```
Your Payout = (Your Winning Shares / Total Winning Shares) × Total Pool
```

Example:

- You own 100 YES shares

- Total YES shares: 1000
- Total pool: 2000 tokens
- Your payout: (100/1000) × 2000 = 200 tokens

Can I sell my shares before resolution?

Not yet. Currently, you can only claim winnings after market resolves. Secondary market trading is planned for future.

AI Resolution

How does AI resolution work?

1. Market expires
2. User triggers AI resolution
3. Tavily AI searches web for real-time context
4. Groq Llama 3.3 70B analyzes and determines outcome
5. AI proposes resolution on-chain
6. 24-hour challenge window begins

Is AI resolution accurate?

AI provides confidence scores (0-100) with reasoning. For objective questions with verifiable data, accuracy is very high (95%+).

What if AI is wrong?

Anyone can challenge the resolution within 24 hours by posting a dispute bond. DAO votes on disputes.

What types of questions work best?

Good:

- "Will Bitcoin reach $100k by Dec 31, 2025?" (verifiable price data)
- "Will Trump win 2024 election?" (official results)
- "Will Ethereum ETF be approved in Q1 2025?" (regulatory announcement)

Bad:

- "Will AI become sentient?" (subjective, no clear criteria)
- "Will world peace be achieved?" (ambiguous definition)

How much does AI resolution cost?

~$0.01 per resolution (Tavily search + Groq inference). Much cheaper than manual resolution

($10-50).

Gasless Transactions

Why do I need gas for BNB but not ERC20 tokens?

ERC-4337 paymasters can sponsor ERC20 token approvals and transfers. Native BNB transfers require gas from sender's account.

What if gasless transaction fails?

The platform automatically falls back from Particle to Pimlico paymaster. If both fail, you'll see an error message.

Can I use my own gas instead?

Yes, toggle off "Use Gasless Transaction" in the UI to pay gas yourself.

How many gasless transactions can I make?

No hard limit per user, but platform has rate limits:

- 50 gasless transactions per user per hour
- 100 per IP per hour

Technical Questions

What blockchain is SeerHive on?

Testnet: BNB Testnet (Chain ID: 97)

Mainnet: BNB Mainnet (Chain ID: 56) - Coming Soon

What is the contract address?

PredictionMarket: `0xc6Dd26D3eE0F58fAb15Dc87bEe3A66896B6D4127`

View on BSCScan

Is the contract audited?

Not yet. Audit planned before mainnet deployment. Use testnet at your own risk.

Can I run my own instance?

Yes! SeerHive is open source. See Getting Started guide.

What APIs does SeerHive use?

- **Groq**: LLM for AI reasoning
- **Tavily**: Web search for real-time data

- **Particle Network**: Paymaster for gasless transactions
- **Pimlico**: Backup paymaster

Troubleshooting

"Transaction failed" error

Possible causes:

1. Insufficient balance
2. Market expired or resolved
3. Network congestion
4. Paymaster rate limit

Solutions:

1. Check token balance
2. Verify market status
3. Wait and retry
4. Try without gasless mode

"Wallet connection failed"

Solutions:

1. Ensure wallet is on BNB Testnet (Chain ID: 97)
2. Refresh page and reconnect
3. Try different wallet (MetaMask, Particle)
4. Clear browser cache

"Contract not found"

Solutions:

1. Verify you're on BNB Testnet
2. Check contract address in config
3. Ensure RPC URL is correct

"Gasless transaction rejected"

Possible causes:

1. Paymaster rate limit exceeded
2. Paymaster out of funds

3. Invalid UserOperation

Solutions:

1. Wait 1 hour and retry

2. Contact team to refill paymaster

3. Try without gasless mode

"AI resolution failed"

Possible causes:

1. API rate limit exceeded

2. Invalid API keys

3. Network timeout

Solutions:

1. Wait and retry

2. Check API key configuration

3. Contact support

Security & Privacy

Is my wallet safe?

Yes. SeerHive never asks for private keys. Wallet connection uses standard Web3 protocols (wagmi, Particle Network).

What data does SeerHive collect?

- Wallet addresses (public)
- Transaction history (on-chain)
- Market interactions (on-chain)

No personal information collected unless you use social login (Particle Network).

Can I remain anonymous?

Yes. Use MetaMask with anonymous wallet. No KYC required.

What if I lose my private key?

If using MetaMask: You lose access to funds (standard crypto wallet risk)

If using Particle Network: Social recovery available through your login method

Roadmap & Features

When is mainnet launch?

Planned for Q2 2026 after:

- Smart contract audit
- Beta testing completion
- Feature polish

What features are coming?

Q1 2026:

- UMA Optimistic Oracle integration
- Challenge/dispute UI
- Copy trading
- Reputation system

Q2 2026:

- Mobile PWA
- Advanced AI (multiple data sources)
- DAO governance
- Liquidity pools

Can I contribute?

Yes! SeerHive is open source. See Contributing guide.

How can I provide feedback?

- GitHub Issues: github.com/abeachmad/seerhive/issues
- Discord: [Coming Soon]
- Twitter: [Coming Soon]

Economics

How does SeerHive make money?

Current: Platform is free (subsidized)

Future:

- Trading fees (1-2%)
- Premium features
- API access fees

What are the costs?

For Users: $0 (gasless transactions)

For Platform:

- Paymaster: ~$0.10 per transaction
- AI Resolution: ~$0.01 per market
- Infrastructure: ~$150/month (testnet)

Is there a token?

Not yet. Token planned for:

- Governance voting
- Staking for dispute resolution
- Fee discounts

Support

How do I get help?

1. Check this FAQ
2. Read Documentation
3. Open GitHub Issue
4. Contact team (Discord/Twitter coming soon)

How do I report a bug?

Open issue on GitHub: github.com/abeachmad/seerhive/issues

Include:

- Description of bug
- Steps to reproduce
- Expected vs actual behavior
- Screenshots (if applicable)

How do I request a feature?

Open feature request on GitHub with:

- Use case description
- Why it's valuable
- Proposed implementation (optional)

Legal

Is prediction market legal?

Depends on jurisdiction. SeerHive is:

- Decentralized (no central authority)
- Testnet only (no real money)
- Educational/experimental

Consult local laws before using with real funds.

What is the license?

MIT License - see LICENSE

Terms of Service?

Coming soon for mainnet launch.

Next Steps

- Getting Started - Start using SeerHive
- Architecture - Understand the system
- API Reference - Explore APIs